src/flightCompV2/PID.cpp

```
/*
 * Program to control ARC-S Servo's with PID logic
 *
 * Name:      Thor Kopenkoskey
 * Created:   March 1, 2025
 * Edited:    March 1, 2025
 */

#include "PID.h"

extern craftState vehicle;
PIDController pid;

void pid_init(PIDController *pid, double Kp, double Ki, double Kd, double integral_min, double integral_max, double output_min, double output_max)
{
    pid->Kp = Kp;
    pid->Ki = Ki;
    pid->Kd = Kd;

    pid->integral = 0.0f;
    pid->prev_error = 0.0f;

    pid->output_min = output_min;
    pid->output_max = output_max;

    pid->integral_min = integral_min;
    pid->integral_max = integral_max;
}

double pid_update(PIDController *pid, double setpoint, double measurement, double dt)
{

  double error = setpoint - measurement; // Where we want to be, minus, where we are. Difference of the two is the needed correction.

  //******Integral******//
  pid->integral += error * dt; // error * dt is a smaller value

  //Clamp integral to not build up and overtake the system with large values
    if (pid->integral > pid->integral_max)
    {
      pid->integral = pid->integral_max;
    }
    if (pid->integral < pid->integral_min)
    {
```

```
45        pid->integral = pid->integral_min;
46      }
47
48    //******Derivative******//
49      double derivative = 0.0;
50      if (dt > 0) //Avoid divide by zero errors
51      {
52        derivative = (error - pid->prev_error) / dt;
53      }
54
55      //Output for PID controller
56      //see https://en.wikipedia.org/wiki/
Proportional%E2%80%93integral%E2%80%93derivative_controller#Control_loop_example
57      // Control function u(t) is Error, integral, and derivative multiplayed by
their gain proportations and then added up
58      double output = (pid->Kp * error) + (pid->Ki * pid->integral) + pid->Kd *
derivative;
59
60      //return current error as prev error for next lop
61      pid->prev_error = error;
62      return output;
63  }
64
```